This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

A Comprehensive Survey on Large Language Model Compression for Artificial Intelligence Applications in Edge Systems

Yuzhu Liang, Changfu Xu, Yaxin Mei, Haodong Zou, Jianxiong Guo, *Member, IEEE*, Xinggang Fan*, Tian Wang, *Senior Member, IEEE* and Haiyang Huang*

***Abstract*—Large Language Models (LLMs) have achieved remarkable performance across various artificial intelligence applications. However, current LLMs cannot be deployed directly on edge nodes due to their large number of parameters. Fortunately, model compression technology has been proposed to reduce the computational workload and memory usage of LLMs, enabling further edge-based LLM services. However, existing research typically concentrates on isolated compression algorithms and lacks a comprehensive perspective on how to leverage these techniques for practical, end-to-end LLM deployment in edge environments. In this survey, we review edge-oriented LLM compression techniques and software–hardware co-design strategies to enable efficient LLM deployment on resource-constrained edge systems and guide future research in this area. First, we analyze techniques for LLM compression from the perspective of cloud–edge collaborative intelligence, including model quantization, parameter pruning, and knowledge distillation. Second, we present several hybrid model frameworks tailored to dynamic, heterogeneous edge environments, based on model architecture, application scenarios, and combination selection. Third, we further refine a four-layer software–hardware co-design and an overhead-aware LLM deployment optimization. Finally, we discuss the challenges of current model compression approaches and offer insights into future research directions, with a focus on edge-based LLM services.**

***Index Terms*—Edge-based LLMs, Model compression, Hybrid compression, Artificial intelligence, and Edge deployment optimization.**

I. Introduction

Large Language Models (LLMs), such as ChatGPT [1] and Gemini [2], are typically deployed as centralized cloud services and have achieved great performance across various Artificial Intelligence (AI) applications [3]. However, with the rapid expansion of industrial Internet of Things (IoT), smart terminals, and vehicular systems, existing cloud-based LLM services cannot meet the low-latency requirements of various AI applications. To address this, cloud-edge collaboration-based LLM services have been proposed that leverage compressed versions of the original models, deployed directly on edge nodes [4]. However, as model scale and computational complexity continue to increase, deploying such LLMs directly on resource-constrained edge nodes poses significant challenges. These challenges are mainly reflected in the following three aspects.

1) **Limited computational resources.** The training and inference of LLMs largely depend on high-performance GPUs/TPUs and require expensive maintenance costs [5]. For example, the inference memory of LLaMA2-7B for processing a 128-token sequence requires 16 GB, which has already exceeded the memory capacities of typical edge devices in the range $[2, 16]$ GB. Even flagship edge platforms such as Qualcomm Snapdragon X Elite, equipped with a 45-TOPS Hexagon NPU, still cannot support full-stack inference, highlighting the mismatch between model scale and edge computing capability [6].
2) **Deployment and execution constraints.** Although edge-cloud collaboration and large model compression techniques have made phased progress in recent years, there are still significant shortcomings in ultra-low-bit quantization, KV Cache usage, and heterogeneous hardware adaptation. For instance, INT8 quantization may reduce energy and area cost compared to FP32 by up to 30× and 116× [7], but often results in noticeable accuracy degradation. In addition, heterogeneous edge hardware leads to inconsistent execution efficiency, hindering portability and stable deployment [8].
3) **Application mismatch.** Generic LLMs usually fail to meet strict requirements in latency, bandwidth, and privacy protection typical of edge scenarios such as healthcare diagnostics and autonomous driving. Transmitting raw data to the cloud not only increases communication cost but also exacerbates privacy risks, motivating the need for lightweight, locally deployable models [9] at the edge.

To address these challenges, we focus on LLM compression as a key enabler for deploying and scaling large AI models on resource-constrained edge nodes, thereby enabling practical

Yuzhu Liang, Yaxin Mei, and Haiyang Huang are with the School of Artificial Intelligence and Computer Science, Nantong University, Nantong, 226019, China; Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519000, China (e-mail: cs_yuzhuliang@163.com, meiyax@163.com, younghuangh@163.com).

Changfu Xu is with the School of Software and IoT Engineering, Jiangxi University of Finance and Economics, Nanchang 330013, China. (e-mail: changfuxu91@outlook.com)

Haodong Zou is with the School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: zou_hd@163.com).

Jianxiong Guo is with the Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China, and also with the Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, Zhuhai 519087, China. (e-mail: jianxiongguo@bnu.edu.cn)

Xinggang Fan is with the Zhijiang College, Zhejiang University of Technology, Shaoxing 312030, China (e-mail: xgfan@zjut.edu.cn).

Tian Wang is with the Engineering Research Center of Cloud-Edge Intelligent Collaboration on Big Data, Ministry of Education, Beijing Normal University, Zhuhai, Guangdong, China; the College of Computer and Data Science, Fuzhou University, China; and the School of Information Engineering, Huzhou University, China; Computer Science and Mathematics, Fujian University of Technology, Fuzhou, China (e-mail: tianwang@bnu.edu.cn).

Yuzhu Liang and Changfu Xu contributed equally to this work.

Corresponding author: Xinggang Fan and Haiyang Huang

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

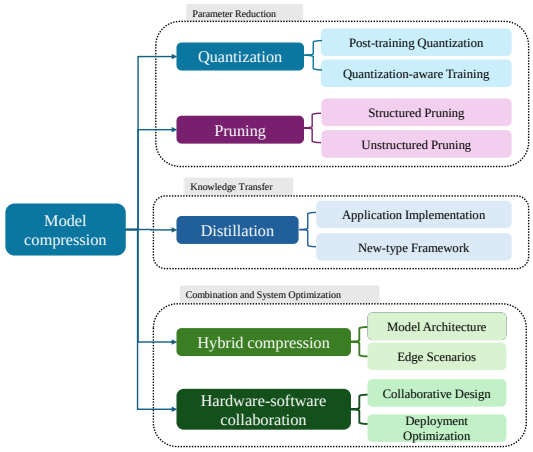


Fig. 1. Taxonomy of large model compression techniques.

edge-based LLM services. In particular, by reducing the computational workload and memory footprint, LLM compression technology enables efficient inference on constrained edge nodes while preserving the performance and privacy of the original LLMs [10]. For instance, existing studies have shown that compressed LLMs can maintain competitive accuracy, reduce resource overhead, and facilitate the shift from cloud-based LLM services to edge-based LLM services, indicating their crucial role in future edge AI systems [11]. To provide a overview of the landscape of LLM compression, Fig. 1 presents a taxonomy that categorizes existing approaches into parameter reduction, knowledge transfer, and hybrid system-level optimization. This taxonomy serves as a conceptual roadmap that guides the organization of the subsequent sections.

To ensure broad coverage and reduce selection bias, we conducted an extensive literature exploration across major digital libraries, including IEEE Xplore, ACM Digital Library, Elsevier ScienceDirect, and arXiv. The search primarily focused on works published between 2018 and 2025, which corresponds to the rapid development period of LLMs and edge AI systems. Representative keywords included combinations of "large language model", "LLM compression", "edge AI", "model quantization", "pruning", "knowledge distillation", "hybrid compression", and "software–hardware co-design". Based on relevance to edge deployment constraints and system-level optimization, over 200 candidate papers were initially screened, from which approximately 110 representative and influential works were selected for detailed analysis and taxonomy construction. Rather than applying strict inclusion–exclusion criteria as in systematic reviews, the selected studies were organized according to compression techniques, architectural characteristics, deployment scenarios, and co-design considerations, aligning with the technical and fast-evolving nature of this survey.

To better structure the survey and guide the subsequent discussions, we focus on the following research questions:

RQ1: What are the core LLM compression techniques that enable deployment under resource-constrained edge and IoT environments?

RQ2: How can quantization, pruning, and knowledge distillation be combined into effective hybrid compression pipelines across different model architectures and edge application scenarios?

RQ3: What software–hardware co-design and deployment optimization strategies are most critical for practical edge-based LLM services, and what open challenges remain?

Although there have been several surveys on LLM compression, they mainly focus on standalone algorithmic designs for LLM compression and lack a comprehensive discussion of edge-based LLM compression, hybrid compression pipelines, and software-hardware collaboration optimization [12]. With the evolution of edge AI toward LLMs, model compression research urgently needs to progress from algorithm-level design to system-level and architecture-level optimization [13]. To address these gaps, this survey provides a comprehensive review of edge-oriented LLM compression techniques, with contributions summarized as follows:

- We comprehensively summarize core LLM compression techniques tailored to diverse edge constraints, complementing existing surveys that pay limited attention to practical edge-based LLM services.
- We provide an in-depth survey of hybrid compression frameworks that integrate quantization, pruning, and knowledge distillation (KD), highlighting their adaptability to different model architectures and AI applications.
- We propose a refined four-layer software–hardware co-design framework spanning the system, model, operator, and instruction levels, offering end-to-end optimization strategies for edge-oriented LLM deployment.
- We analyze key technical challenges arising from edge constraints and outline future research directions that can guide the evolution of edge-based LLM services in this rapidly developing field.

As shown in Fig. 2, the overall structure of this survey is organized as follows: Sec. II reviews foundational compression techniques; Sec. III discusses hybrid methods and scenario-driven combinations; Sec. IV presents software–hardware co-design and deployment optimization; Sec. V highlights future research directions; and Sec. VI concludes this survey.

II. Foundational Compression Techniques

Within cloud-edge-device collaborative architectures, model compression serves not only as a computational optimization strategy but also as a crucial enabler for the paradigm of "cloud-based training and edge-side inference". Current mainstream compression methods fall into three categories: *quantization*, *parameter pruning*, and *KD*. These approaches differ in theoretical basis and deployment characteristics: quantization reduces model size by lowering numerical precision; pruning eliminates redundant connections to reduce computation; and KD transfers knowledge to smaller models while preserving task performance. This section provides a comprehensive overview of their core mechanisms, progress in edge-oriented deployment, and remaining challenges.

From an edge deployment perspective, not all LLM compression methods are suitable for practical use. Many standalone compression approaches are primarily designed for

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

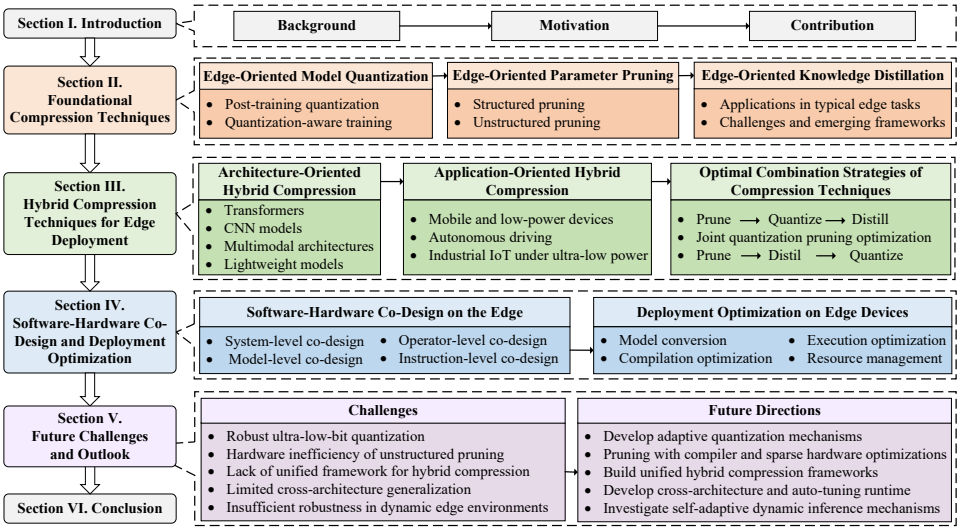


Fig. 2. The overall structure of this survey.

cloud or data-center environments, where abundant computation, memory, and stable power supply are assumed [14]. Such methods often rely on extensive retraining, large calibration datasets, or homogeneous hardware platforms, which limits their portability and practicality in heterogeneous and resource-constrained edge systems. Accordingly, this survey focuses on compression techniques that explicitly account for edge deployment constraints.

A. Edge-Oriented Model Quantization

Model quantization converts high-precision weights and activations (e.g., FP32) into low-bit representations (e.g., INT8, INT4, or 1-bit) [15], aiming to reduce memory footprint and computational cost while retaining acceptable accuracy. Based on whether re-training is required, quantization can be categorized into *post-training quantization* (PTQ) and *quantization-aware training* (QAT). These two paradigms exhibit distinct performance and deployment trade-offs in edge scenarios:

1) **PTQ.** PTQ directly converts pre-trained model parameters into lower-bit formats [16], including statistic-driven static quantization (e.g., round-to-nearest, RTN) and mixed-precision quantization designed for LLMs with outlier activations (e.g., LLM.int8()) [17]–[19]. Its major advantage lies in deployment simplicity and the elimination of retraining. However, aggressive low-bit quantization typically introduces significant accuracy degradation. Recent studies propose advanced PTQ methods based on error minimization and data reconstruction to mitigate this issue [20], [21].
2) **QAT.** QAT incorporates quantization effects into the training process [22]. A typical approach inserts pseudo-quantization nodes and leverages straight-through estimators to simulate quantization behavior [23]. By explicitly modeling quantization noise during training, QAT significantly improves low-bit precision. Nevertheless, the retraining cost makes QAT unsuitable for resource-constrained on-device learning in IoT systems.

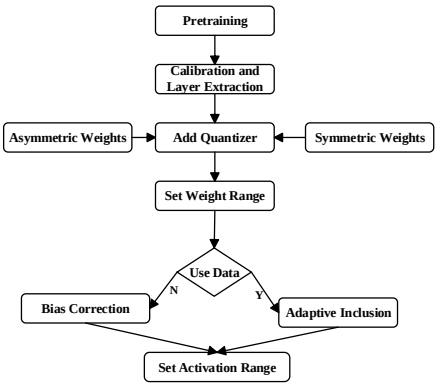


Fig. 3. Overview of the PTQ workflow.

In general, PTQ has become the mainstream solution for deploying large-scale models on edge devices due to its low deployment overhead [24], [25]. Nevertheless, ensuring inference speed advantages while minimizing accuracy degradation remains a central research challenge. Table I summarizes the differences in precision and deployment characteristics between these two quantization strategies.

It is worth noting that the relative suitability of PTQ and QAT for edge deployment primarily depends on deployment-stage constraints rather than compression effectiveness. Although QAT generally achieves higher accuracy under low-bit quantization, it requires retraining or fine-tuning with data and substantial computational resources, which are often unavailable in practical edge scenarios. In contrast, PTQ can be applied directly to pretrained models without retraining, making it more favorable for rapid and resource-constrained edge deployment. Detailed comparisons between PTQ and QAT are provided later in this section.

1) PTQ: Fig. 3 illustrates the overall workflow of PTQ. Starting from a pre-trained model, PTQ extracts calibration statistics to estimate data distributions, inserts quantizers into

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE I
COMPARISON BETWEEN PTQ AND QAT

Category	PTQ	QAT
Accuracy loss [26]	2–5%	$< 2\%$
Hardware requirement [26]	General-purpose hardware	Requires quantization-aware support
Training cost [26]	None	High
Compression ratio [26]	Up to $4\times$	Up to $8\times$
Typical deployment scenario [26]	Mobile / edge devices	Cloud or training-capable servers
Representative methods	SmoothQuant [11], LLM-FP4 [27]	CBQ [28], LLM-QAT [29]

the network, and sets appropriate weight and activation ranges. High-precision parameters are then mapped to low-bit representations, and further optimized through methods such as bias correction or adaptive rounding, thereby enabling efficient compression and lightweight inference on edge devices.

However, as model architectures become increasingly complex, conventional PTQ techniques struggle to handle activation outliers in LLMs, module-wise sensitivity variations in vision transformers (ViTs), and domain shift during cross-domain deployment. Consequently, recent research has shifted from generic low-bit compression toward *structure-aware and scenario-adaptive PTQ*, especially for ultra-low-bit (4-bit or below) quantization and multimodal models. In such settings, mitigating accuracy degradation becomes crucial for deploying LLMs on heavily resource-constrained IoT and edge devices. Representative PTQ works are summarized in Table II, with notable directions outlined below:

1) **Handling activation outliers.** SmoothQuant [11] migrates scaling factors from activations to weights, enabling full W8A8 quantization of OPT-175B. On an edge-grade RK3588 CPU, SmoothQuant yields $1.56\times$ inference acceleration, 75% memory reduction, and $<1\%$ accuracy loss. However, its applicability remains limited to INT8 without exploring lower-bit formats.
2) **Extreme low-bit quantization.** LLM-FP4 [27] introduces a 4-bit floating-point format designed to mitigate distribution shifts. For GPT-3 1.3B, it achieves nearly 80% of FP16 performance using only 0.5% calibration data, though its behavior for streaming or long-context inference remains unverified. SmoothQuant+ [30] supports 4-bit PTQ for visual inference engines, achieving 1.9–$4.0\times$ higher throughput than FP16 deployed on dual A100 GPUs, demonstrating engineering potential for extreme quantization.
3) **Multimodal optimization.** CalibQuant [28] proposes a 1-bit KV-cache quantization scheme for multimodal LLMs by optimizing channel scaling and attention calibration. On InternVL-2.5, it delivers a $10\times$ throughput boost and $16\times$ memory reduction, achieving higher CIDEr scores than full precision. Stability under long-context scenarios, however, remains unexplored.
4) **ViT module-aware quantization.** PTQ4ViT [31] performs hierarchical quantization with channel-adaptive calibration to handle module sensitivity in ViTs and is optimized for FPGA deployment. On DeiT-S, 4-bit quantization reduces edge-side inference latency by 62% and compression by $8\times$ with only 0.8% ImageNet Top-1 accuracy loss, but its performance degrades significantly below 4-bit.
5) **Cross-domain adaptation.** DAQuant [29] addresses domain shift in low-bit inference by introducing domain-adaptive calibration. On ImageNet and COCO, it constrains accuracy degradation to 1% and delivers $4\times$ compression while meeting real-time edge-inference requirements. However, insufficient hardware validation and compatibility issues remain for ultra-low-bit settings.

Although these methods demonstrate promising compression results across modalities and architectures, PTQ still faces challenges such as accuracy loss, hardware incompatibility, and heavy dependence on model structure. For instance, most edge accelerators support only INT8 arithmetic, lacking native support for sub-8-bit formats. Research on PTQ for convolutional and recurrent networks also remains limited. In summary, PTQ for edge inference is shifting from 8-bit toward 4-bit and even 1-bit quantization, with the core challenge to minimize accuracy loss while maintaining inference speed.

2) QAT: Unlike PTQ, QAT adopts a "train-and-compress" paradigm by injecting quantization operations during training, enabling the model to adapt to accuracy degradation caused by low-bit representations. As a result, QAT generally provides superior performance in terms of compression ratio, inference efficiency, and accuracy preservation for edge-oriented deployment. Fig. 4 illustrates the overall workflow of QAT. Starting from a pre-trained model, QAT performs module fusion and inserts pseudo-quantization nodes to simulate quantization effects. The model is then fine-tuned on real data to mitigate precision loss. Finally, the pseudo-quantized model is converted into an integer form to produce a lightweight model that balances accuracy and deployment efficiency.

From a performance perspective, QAT can typically limit accuracy degradation within 2% and achieve up to an $8\times$ compression ratio. For example, CBQ, jointly proposed by Huawei and USTC, achieves $7\times$ compression with 99% accuracy retention using only 0.1% training data [32]. Despite these advantages, QAT incurs high training costs and complex implementation requirements, typically requiring full training datasets and powerful compute resources, thereby restricting its applicability to IoT devices.

The scope of application of QAT has expanded beyond early convolutional networks (CNNs) to include ViTs, LLMs, and multimodal models. Notably, Google's Gemma 3 reduces VRAM usage from 54 GB (BF16) to 14.1 GB (INT4) using QAT [33]. Current research trends mainly focus on three directions: (1) simulating quantization behavior during train-

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE II
SUMMARY OF REPRESENTATIVE PTQ METHODS FOR EDGE-ORIENTED LLM DEPLOYMENT

Method	Technical Innovation	Quantization Effect	Limitation
SmoothQuant [11]	Shifts scaling factors to weights to mitigate activation outliers	$1.56\times$ speedup and 50% memory reduction on OPT-175B with $< 1\%$ accuracy drop	Limited to INT8; no exploration of lower-bit settings
LLM-FP4 [27]	Designs a 4-bit floating-point format (FP4) to alleviate distribution shift	GPT-3 1.3B reaches $\sim 80\%$ of FP16 performance using only 0.5% calibration data	Limited robustness for streaming inference and long-context scenarios
SmoothQuant [30]	Efficient 4-bit weight-only PTQ achieving lossless compression	Improves throughput by 1.9–$4\times$ compared with FP16 deployed on A100 GPUs	Dependent on specific GPUs and custom CUDA kernels
CalibQuant [28]	1-bit KV-cache quantization with channel scaling and attention calibration	Achieves $10\times$ throughput and $16\times$ memory reduction on InternVL-2.5	Stability in long-context multimodal tasks remains unverified
PTQ4ViT [31]	Hierarchical quantization with channel-adaptive calibration for ViTs	62% latency reduction at edge deployment; ImageNet Top-1 loss only 0.8% at 4-bit	Severe degradation below 4-bit; near-lossless only at 6/8-bit
DAQuant [29]	Domain-adaptive calibration to address cross-domain data shift	$< 1\%$ accuracy loss on ImageNet/COCO with $4\times$ compression	Hardware compatibility issues for low-bit settings; lacks extensive real-device tests

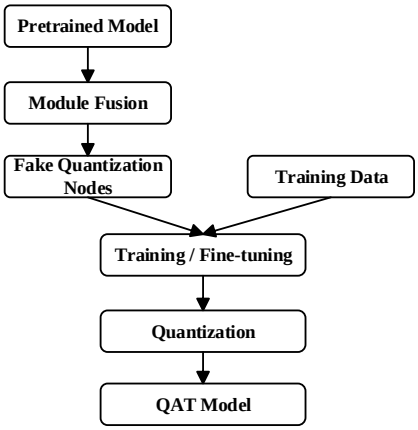


Fig. 4. Workflow overview of QAT.

ing, (2) designing quantization-aware loss functions, and (3) optimizing training strategies to improve low-bit performance under diverse deployment constraints. Representative QAT works are summarized in Table III, with notable achievements below:

1) **Novel QAT training paradigms.** LLM-QAT [21] integrates QAT with parameter-efficient fine-tuning (PEFT) for 4-bit deployment of LLaMA-7B, achieving a $2\times$ inference speedup. EfficientQAT [34] tackles high QAT training cost using a lightweight two-stage approach, enabling 2-bit quantization of LLaMA-2-70B on a single A100 GPU. Its 4-bit version achieves 69.48% on MMLU, outperforming uniform quantization by 5%.
2) **Mobile and edge-oriented QAT.** Squat [35] uses entropy-based alignment to address both sub-8-bit quantization and edge instruction compatibility, achieving 69.4% accuracy on BLiMP without fine-tuning and $2.37\times$ inference acceleration. Agile-Quant [36] proposes dynamic activation quantization guided by token importance, which alleviates attention-layer outliers and enables $2.55\times$ acceleration on Raspberry Pi with 4-bit weight and activation quantization.
3) **Ultra-low-bit QAT for BERT.** Q-BERT [37] leverages Hessian-based sensitivity metrics to support extreme low-bit (e.g., 2-bit) quantization, significantly reducing model footprint for mobile and embedded deployment. Further work [38] compresses MobileBERT and quantizes it for real-time tweet analysis on edge devices, shrinking the model by $160\times$ with only a 4.1% drop in accuracy, demonstrating the practical feasibility of QAT under extreme compression.

Although QAT shows promising results across low-bit deployment, hardware adaptation, and performance trade-offs, several challenges remain: QAT models may suffer unstable accuracy under low-bit activation quantization, exhibit strong dependence on specific hardware architectures, and lack optimization coverage for traditional network structures. Moreover, extremely low-bit quantization still risks accuracy degradation in complex tasks. Overall, QAT is evolving toward lower-bit formats, broader architecture compatibility, and tighter software–hardware co-design. The core challenge in edge AI is maintaining accuracy, stability, and generalization under extreme compression and heterogeneous hardware constraints.

B. Edge-Oriented Parameter Pruning

Parameter pruning reduces model size and computation by removing redundant parameters or structures. Traditionally, pruning was mainly applied to reduce model complexity and improve generalization under cloud or HPC environments [39]. In contrast, pruning for edge AI increasingly targets *real-time responsiveness, low energy consumption, and hardware adaptability*. Due to strict constraints on memory bandwidth, compute throughput, and power budget in edge devices, current efforts have shifted from achieving extremely high sparsity toward *hardware-effective acceleration*.

Unlike quantization, which compresses numeric representation, pruning directly removes weights. Based on granular-

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE III
SUMMARY OF REPRESENTATIVE QAT METHODS FOR EDGE DEPLOYMENT

Method	Technical Innovation	Quantization Effect	Limitation
LLM-QAT [22]	Trains without real data by using generated synthetic data	$< 1\%$ accuracy loss for BERT under 4-bit quantization	Only supports 4-bit weight quantization
EfficientQAT [34]	Dynamically adjusts gradient magnitude and learning rate during QAT	Reduces training time by 40%; 4-bit accuracy drop of only 0.5%	Strongly depends on high-quality, diverse datasets
Squat [35]	Automatically adapts quantization bit-width based on data distribution	Reduces latency by 50% and achieves a $3\times$ energy efficiency improvement on edge devices	Supports only ARM architecture; lacks support for generative tasks
Agile-Quant [36]	Proposes a dynamic activation quantization strategy	Enables simultaneous 4-bit quantization of weights and activations	Severe performance degradation under 4-bit activation quantization
Q-BERT [37]	Hessian-based grouped quantization for compression-oriented QAT	Compresses model parameters by $13\times$; embeddings and activations compressed up to $4\times$	Supports only encoder models; not applicable to mainstream LLMs
MobileBERT [38]	Framework for evaluating TensorFlow Lite quantized MobileBERT on edge devices	$160\times$ size reduction; 4.1% accuracy loss; runtime $1.15\times$ slower than BERT Large	Validated only on a single model, task, hardware platform, and quantization configuration

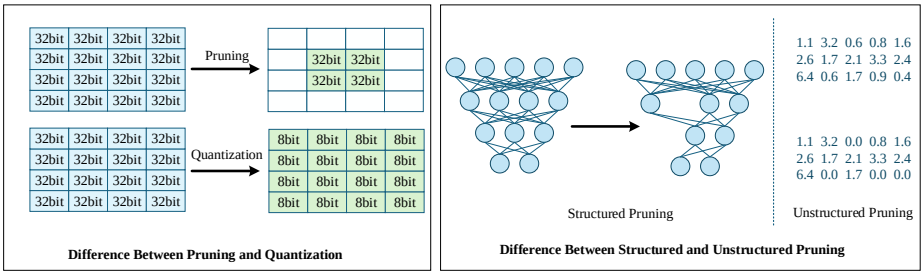


Fig. 5. Differences between pruning (structured/unstructured) and quantization.

TABLE IV
COMPARISON BETWEEN STRUCTURED AND UNSTRUCTURED PRUNING

Category	Structured / Unstructured Pruning
Compression ratio [40]	20-70% / 50-90%
Accuracy recovery cost [40]	Low (minor fine-tuning) / High (retraining required)
Hardware compatibility [40]	Strong (general GPUs/NPUs) / Weak (specialized sparse engines)
Granularity [40]	Channels/layers/heads / Individual weights
Typical methods	SlimLLM [13], DarwinLM [39] / UnIT [41], Mustafar [42]

ity, pruning can be categorized into structured and unstructured pruning. Structured pruning removes regular units (e.g., channels, heads, layers) and offers high hardware compatibility suitable for general-purpose edge hardware. Unstructured pruning eliminates individual weights, achieving higher compression ratios but requiring specialized accelerators. The conceptual difference between pruning and quantization, and between structured vs. unstructured pruning, is shown in Fig. 5, and key distinctions are summarized in Table IV.

1) Structured Pruning: With the rising demand for edge deployment, structured pruning has attracted attention due to its strong hardware friendliness. The research focus has evolved toward two fundamental challenges: *low-cost accuracy recovery after pruning* and *ensuring that pruned architectures are efficiently executable on edge hardware*. Recent studies explore pruning mechanisms, cost reduction through fine-tuning, and hardware-aware designs. Representative works are summarized in Table V.

1) **Reducing fine-tuning cost.** DarwinLM [39] proposes dynamic pruning based on evolutionary search, achieving a 40% parameter reduction on a 175B model with $< 1.5\%$ accuracy loss. Notably, it requires only 10B tokens for recovery, nearly $1/5$ of conventional cost, and provides 1.5–$2\times$ CPU inference speedup. FASP [43] reduces fine-tuning overhead by inheriting cross-layer weights, reaching 60% sparsity on GPT-3 with a 40% memory reduction while keeping accuracy loss within 1.5%.
2) **Improving real-time inference on edge devices.** TransAct [44] prunes transitional activations in Transformers, shrinking KV cache by 75% and reducing latency to 14 ms under $< 3\%$ loss, enabling long-text generation at the edge. In task-specific pruning, SepPrune [45] targets speech separation, achieving $2\times$ acceleration and < 500 MB of memory usage on the Raspberry Pi 4B, thereby satisfying wearable and interactive edge deployment.
3) **Post-pruning recovery optimization.** Layer-wise L2 norm pruning [46] compares fine-tuning strategies and demonstrates that KL-based self-distillation (L2PSD) outperforms conventional L2PFT in accuracy, data ef-

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE V
SUMMARY OF STRUCTURED PRUNING METHODS FOR EDGE DEPLOYMENT

Method	Technical Innovation	Pruning Effect	Application Scenario
SlimLLM [13]	Differentiable scoring with joint pruning and tuning	Retains 90% performance of LLaMA-13B; 50% VRAM reduction	Resource-limited edge devices
DarwinLM [39]	Evolutionary dynamic pruning	40% reduction on 175B with $< 1.5\%$ loss	Lightweight LLM deployment
FASP [43]	Cross-layer weight inheritance	60% sparsity, 40% memory reduction	Edge-side weight inheritance
TransAct [44]	Task-agnostic transformer activation pruning	75% KV reduction; latency $<$ 20ms	Long-text generation on edge
SepPrune [45]	Frequency-domain pruning for speech tasks	$2\times$ speedup on Pi 4B	Real-time speech on wearables
L2 norm Pruning [46]	KL-based self-distillation post-prune	L2PSD surpasses L2PFT in efficiency	Offline QAT under edge constraints

TABLE VI
SUMMARY OF UNSTRUCTURED PRUNING METHODS FOR EDGE DEPLOYMENT

Method	Technical Innovation	Deployment Effect	Scenario
AST [47]	Learnable masks with KD + SLoRB	Minimal loss using 0.4% pre-training resources	Knowledge-intensive LLM tasks
MaskLLM [48]	N:M sparsity via Gumbel sampling	6.72 perplexity without tuning	End-to-end learning
UnIT [41]	Dynamic inference-time pruning	82% latency reduction, 84% energy savings	MCU deployment
UnIT Pruner [49]	Energy-accuracy balanced pruning	74–96% energy/time reduction	Low-maintenance devices
Mustafar [42]	KV cache pruning for LLMs	45% memory reduction, $2.23\times$ throughput	Long-text mobile inference
Mamba Pruning [50]	State-transition pruning for Mamba	Efficient long-sequence edge inference	Sequence modeling

ficiency, and computational cost, particularly for offline edge scenarios such as on-device QA.

4) **Hardware-aware pruning.** SlimLLM [13] jointly optimizes pruning and fine-tuning using differentiable structural scoring, retaining 98.7% performance at 20–30% pruning with $> 60\%$ model size reduction. It achieves $2.55\times$ acceleration on Raspberry Pi without retraining, laying groundwork for prune–quant co-design in edge settings.

2) Unstructured Pruning: Unstructured pruning traditionally suffers from irregular sparsity patterns, making it difficult for edge hardware to utilize. To address this bottleneck, recent advances focus on semi-structured design, dynamic inference pruning, diverse pruning targets, and architecture specialization. Table VI summarizes related work.

1) **Semi-structured sparsity with accuracy retention.** AST [47] combines KD and SLoRB for learnable masks, requiring only 0.4% of the pre-training resources of LLaMA2-7B while minimizing accuracy loss. MaskLLM [48] models N:M sparsity with Gumbel sampling, achieving 6.72 perplexity without tuning, supporting end-to-end learning tasks.
2) **Dynamic inference pruning for low power.** UnIT [41] enables runtime pruning for microcontrollers, achieving 82.03% latency reduction and 84.38% energy savings on MSP430. UnIT Pruner [49] extends it by balancing energy and accuracy, reducing inference time by 74.4–94.7% and energy by 74.2–96.5%, enabling long-term low-maintenance edge operation.
3) **Beyond static weights.** Mustafar [42] prunes dynamic KV-cache rather than parameters, reducing memory use by 45% at 70% sparsity with a $2.23\times$ throughput increase, addressing LLM deployment bottlenecks for long-text mobile inference.
4) **Architecture specialization.** For Mamba architectures, [50] prunes state-transition parameters to enable long-sequence processing on constrained edge devices, thereby reducing computational and memory overhead.

Unstructured pruning still faces a fundamental barrier in edge deployment due to limited hardware compatibility. Because pruning produces irregular sparse matrices, most general-purpose computing frameworks and hardware cannot efficiently exploit sparsity, preventing theoretical computational reduction from translating into practical inference acceleration. This limitation is widely regarded as the "fatal bottleneck" of unstructured sparsity. In addition, excessive sparsity may weaken the model's representational capacity, leading to a loss of accuracy that often requires expensive fine-tuning or weight "regrowth" techniques to recover. To address these issues, recent efforts aim to mitigate irregularity through semi-structured sparsity [47], [48] and enhance utilization via inference-time dynamic pruning [41]. Future directions call for tighter software–hardware co-design, such as enabling flexible semi-structured sparsity patterns [51] or building dedicated sparse computing engines [52] to achieve practical speedups on edge devices under high compression ratios.

In summary, unstructured pruning offers superior compression but depends heavily on hardware support and complex recovery processes, whereas structured pruning is more hardware-friendly but entails trade-offs between computational efficiency and model capacity. Hybrid pruning frameworks, such as the "structured-then-unstructured" approach proposed in STUN [53], may overcome these limitations. With further integration of dynamic pruning and diversified pruning targets, hybrid sparsity is expected to become a more powerful pruning paradigm for edge AI.

C. Edge-Oriented KD

KD aims to improve model efficiency by transferring knowledge from a large teacher model to a smaller student model while minimizing performance degradation. The teacher–student paradigm is shown in Fig. 6. A high-capacity teacher network produces reference logits, while multiple

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

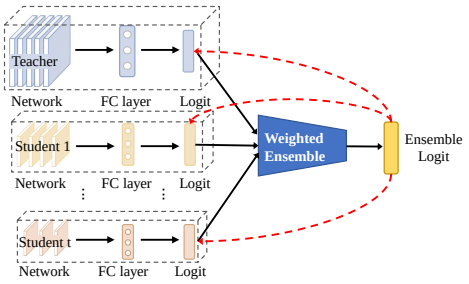


Fig. 6. Teacher-student KD framework.

student networks generate their own logits through lightweight backbones followed by FC layers. The student outputs are aggregated by a weighted ensemble module to form an ensemble logit, which serves as the distillation target. During training, the students are jointly optimized by aligning their predictions with the ensemble and teacher outputs, whereas only the student networks are retained for inference.

Over the past few years, KD has been widely used in computer vision, NLP, and multimodal learning, leading to several comprehensive surveys. For example, [54] summarizes semantic- and causal-level knowledge transfer from a cross-modal perspective, [55] demonstrates that distilling models using synthetic compressed datasets can reduce training iterations by 60%, and [56] focuses on distillation for object detection tasks. However, these works mainly explore methodological or task-specific aspects and overlook rapidly emerging KD research targeting edge deployment. Therefore, this section discusses recent advances and challenges in KD for edge-oriented applications and introduces new frameworks designed to improve edge efficiency.

1) Applications in Typical Edge Tasks: With the increasing demand for deploying models on resource-constrained devices, KD has been widely adopted to compress deep neural networks for edge inference. For example, [57] introduces a KD-based diagnostic system that significantly compresses medical models and accelerates inference on edge devices. Similarly, [58] applies KD to federated learning, reducing communication cost by $250\times$ and improving cloud-side accuracy.

Despite these gains, KD still faces challenges when deployed in computationally intensive edge scenarios, such as vision, detection, and vehicular networks. Recent advances have tackled these challenges, as summarized in Table VII. Representative examples include:

1) **Vision foundation models on edge.** KD is increasingly used to adapt large vision foundation models (LVFMs) to edge-friendly architectures [59]. CustomKD [60] mitigates architectural disparities between LVFMs and lightweight edge backbones, significantly improving MobileNetV3 performance for unsupervised domain adaptation (OfficeHome, DomainNet), CIFAR-100, and ImageNet. VL2Lite [61] distills directly from pretrained vision–language models without teacher training, boosting fine-grained classification performance on ResNet-18 and MobileNet-V2 without increasing inference cost.
2) **Object detection on constrained devices.** KD helps deploy advanced detectors on edge devices. Setyanto et al. [62] introduce a lightweight distillation framework for YOLO-based models, using CSPDarknet53 as the teacher and MobileNetV2/RepViT as students. Their variants, M-YOLO-CRD and RV-YOLO-CRD, reduce model size by $< 30\%$ while maintaining acceptable mAP ($< 4\%$ degradation), making them feasible for edge hardware.
3) **Vehicular networks and privacy-aware edge AI.** KD has shown potential in distributed, privacy-sensitive environments. Pea-KD [63] combines parameter shuffling and teacher-prediction pretraining to enhance small BERT models in heterogeneous vehicular scenarios, outperforming seven baseline methods (including FedAvg), improving accuracy by 4.4% while preserving privacy.

It is worth clarifying that, in KD-based object detection frameworks such as YOLO distillation, the reported reduction in model size refers to the deployment of a predefined lightweight student architecture (e.g., MobileNetV2 or RepViT), rather than a structural modification of the teacher model itself. Knowledge distillation does not intrinsically reduce model parameters; instead, it enables compact student models to approximate the performance of large YOLO-based teachers, thereby allowing significantly smaller models to be deployed on resource-constrained edge devices.

2) From Recent Advances to Open Challenges in Edge-Oriented KD: Recent studies have significantly advanced KD for edge-oriented deployment by adapting teacher–student paradigms to heterogeneous architectures, task characteristics, and resource constraints. As reviewed in the preceding subsection, architecture-aware distillation, privacy-preserving and data-free KD, as well as task-specific KD for vision, detection, and vehicular scenarios, have demonstrated that KD can effectively reduce model complexity while maintaining acceptable performance on resource-constrained edge devices. These advances indicate that several classical limitations of KD, such as architectural mismatch and excessive training overhead, can be partially alleviated through careful design.

Nevertheless, despite these improvements, several fundamental challenges remain unresolved in practical edge settings. First, the student capacity gap continues to limit distillation effectiveness [64]. Although recent methods mitigate architectural incompatibility between large foundation models and lightweight students, the intrinsic representational constraints of edge-friendly models often prevent them from fully approximating teacher behavior under strict memory and computation budgets, resulting in a performance ceiling.

Second, soft-label bias amplification persists in KD-based edge systems [65]. While existing approaches improve efficiency and accuracy, biases embedded in teacher predictions—particularly in data-imbalanced or recommendation-oriented IoT applications—may be propagated or amplified during distillation. Most current KD frameworks lack explicit mechanisms to regulate or debias transferred knowledge, potentially degrading fairness and robustness in long-term deployment.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE VII
SUMMARY OF KNOWLEDGE DISTILLATION METHODS FOR EDGE DEPLOYMENT

Method	Technical Innovation	Research Outcome	Applicable Scenario
Medical DNN Compression [57]	KD-based model for automated disease diagnosis	$6.14\times$ and $5.86\times$ acceleration	Fog/edge devices
CustomKD [60]	Resolves architecture mismatch between LVFMs and edge backbones	Significant gains on UDA, CIFAR-100, and ImageNet	Lightweight edge models
VL2Lite [61]	Distillation from pretrained VLMs without training teacher	Improves ResNet-18 and MobileNet-V2 by 6.43% and 6.47%	Resource-constrained edge devices
M-YOLO-CRD [62]	Lightweight distillation for YOLO-based detectors	$< 30\%$ size reduction, $< 4\%$ mAP drop	Edge object detection
Pea-KD [63]	Parameter shuffling + teacher prediction pretraining	4.4% accuracy improvement; supports Transformer-based models	Wearables, IoT devices

Third, indiscriminate knowledge transfer remains a common limitation. Many KD methods transfer teacher outputs uniformly, introducing redundant or task-irrelevant information that increases optimization difficulty for compact students. Although task-customized and selective distillation strategies have been explored, fine-grained control over knowledge selection under edge constraints is still limited [66].

To address these residual challenges, a series of emerging KD frameworks have been proposed. Elastic student KD temporarily expands the student model during training to enhance representational capacity and then restores it to a compact form for inference, effectively mitigating capacity limitations without increasing deployment overhead. Data-free and privacy-enhanced KD frameworks decouple distillation from raw data access, improving suitability for privacy-sensitive edge environments [63]. In addition, architecture-aware and task-adaptive KD paradigms aim to enable selective knowledge transfer that aligns with both hardware characteristics and application semantics [60].

In summary, recent advances have substantially improved the practicality of KD for edge AI, yet they do not fully resolve challenges related to student capacity, bias propagation, and knowledge selectivity [67]. Emerging KD frameworks should therefore be regarded as extensions that address the remaining gaps under extreme resource constraints, reinforcing KD's role as a key enabler of efficient, reliable, and scalable edge AI [68].

III. Hybrid Compression Techniques for Edge Deployment

Model compression aims to reduce model parameters, computation, and memory footprint to fit resource-constrained environments. However, each standalone technique has intrinsic limitations. Quantization reduces storage and computation by lowering the numerical precision of weights and activations, but quantization noise can degrade layers with high numerical sensitivity, leading to accuracy degradation [69]. Although pruning effectively reduces structural redundancy, excessive sparsity can diminish representation capacity, and identifying important channels or parameters remains challenging for large networks [70]. KD can compensate for performance loss caused by compression, yet the student model is constrained by its limited capacity and may inherit biases from the teacher's predictions, thereby harming generalization [71].

Therefore, relying on a single compression method often fails to balance extreme compression and performance retention. Hybrid compression techniques have emerged to overcome these bottlenecks. Rather than merely stacking methods, hybrid compression strategically integrates pruning, quantization, and distillation under a collaborative framework, leveraging complementary strengths to achieve a "1+1>2" effect. This section comprehensively reviews hybrid compression from three perspectives: architecture- and application-oriented approaches, as well as optimal combination strategies, to provide a comprehensive roadmap for efficient edge deployment.

A. Architecture-Oriented Hybrid Compression

Different model architectures exhibit unique computational patterns and critical components, necessitating tailored hybrid compression strategies that align with hardware characteristics to maximize efficiency without compromising core functionality. The prevailing approaches for dominant architectures are as follows:

1) **Transformers.** While multi-stage hybrid compression pipelines are not inherently specific to Transformer-based models, a common strategy in Transformer compression instantiates a general hybrid framework using a "3-stage pipeline" of structured pruning, LoRA-based fine-tuning, and quantization. This design is motivated by the large parameter redundancy and structured computation patterns of self-attention layers, rather than by model exclusivity. For example, LLM-Pruner employs structural pruning coupled with LoRA recovery; for LLaMA-7B, it reduces parameters by 20% with minimal accuracy loss (retaining $\sim 95\%$ performance) [72]. While this reduction is substantial, the absolute parameter count of LLaMA-7B remains large, which may limit its direct deployment on resource-constrained edge devices. Therefore, such methods are more relevant for server-assisted or high-end edge scenarios (e.g., AI co-processors or GPU-enabled devices) rather than ultra-low-power devices. Furthermore, it demonstrates compatibility with INT8 quantization, enabling significantly reduced memory usage on resource-constrained devices [73].
2) **CNN models.** Hybrid designs often integrate pruning, quantization, and knowledge distillation to maximize

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

model efficiency. For instance, the APQ framework jointly optimizes pruning and quantization policies, achieving an 8× reduction in BitOPs for ResNet-34 on ImageNet without loss in accuracy relative to the 8-bit baseline [74]. Similarly, PQK explicitly combines these three techniques into a unified pipeline, demonstrating effective compression capabilities for image recognition tasks [75].

3) **Multimodal architectures.** Hybrid compression pipelines are increasingly adopted to optimize vision-language models. For instance, TinyCLIP integrates cross-modal knowledge distillation with weight pruning, transferring semantic affinities from a large teacher (e.g., ViT-L/14) to a pruned student, reducing model size by 50% while maintaining comparable zero-shot performance [76]. Furthermore, MobileVLM combines a lightweight architecture (equivalent to structured pruning) with knowledge distillation and is optimized for mobile deployment (often involving quantization). For example, in the reported implementation, its 1.4B-parameter variant achieves a token generation rate of 21.5 tokens per second on a Snapdragon 888 processor, resulting in an end-to-end latency of approximately 18.5 seconds to generate 256 tokens [77]. This indicates that, while hybrid compression enables mobile deployment, larger multimodal models may still incur substantial latency on edge devices.
4) **Lightweight models.** Advanced co-design strategies now integrate structural re-parameterization, KD, and depthwise separable convolutions to further compress model size and enhance execution efficiency. For example, RepViT revisits mobile CNN architectures, achieving over 80% Top-1 accuracy on ImageNet with a latency of just 1.0 ms on an iPhone 12, significantly outperforming previous state-of-the-art lightweight models [78].

Overall, architecture-oriented hybrid compression enables models to maintain high task performance while achieving substantial structural reduction and low computation overhead. This technique serves as a crucial bridge for deploying LLMs across heterogeneous edge systems. It is worth noting that the above Transformer-oriented pipeline represents an architecture-aware instantiation of hybrid compression, and aligns with the general multi-stage compression strategies discussed later in this section, rather than constituting a Transformer-exclusive paradigm.

B. Application-Oriented Hybrid Compression

In real-world edge AI systems, compression requirements vary across latency, energy consumption, model size, and reliability. These systems often operate as cyber–physical systems (CPS), where sensing, intelligent inference, and actuation form a closed physical–digital–physical (PDP) loop. Within this loop, LLM compression is not merely a means of reducing computational cost, but a key enabler for meeting application-specific system constraints. Hybrid compression is thus customized according to specific deployment goals and system constraints.

1) **Mobile and low-power devices.** This category typically corresponds to time- and delay-sensitive PDP loops, where rapid digital inference is required to provide timely feedback or decision support to the physical world. Strategies for edge deployment often orchestrate knowledge distillation, quantization, and compact architecture design (serving a similar role to structural pruning) to meet strict hardware constraints. For instance, TinyVQA integrates supervised knowledge distillation with 8-bit quantization to deploy a multimodal model on a GAP8 microprocessor; it reduces the model size to just 339 KB while achieving 79.5% accuracy and a low latency of 56 ms for post-disaster assessment tasks [79]. From the PDP-loop perspective, such compression enables the digital intelligence layer to satisfy stringent latency and memory constraints imposed by the physical environment.
2) **Autonomous driving.** Autonomous driving represents a critical safety- and loss-sensitive PDP loop, in which perception and decision errors in the digital domain may directly lead to severe physical consequences. Safety-critical perception requires minimal latency under strict reliability constraints. To address this, QD-BEV introduces a quantization-aware view-guided distillation framework for multi-view 3D object detection. By combining QAT with cross-view distillation, it achieves a 5.9× reduction in BitOPs and 2.9× lower latency on edge GPUs compared to the full-precision baseline, while preserving high detection accuracy in complex driving scenarios [80]. This example illustrates that hybrid compression can improve efficiency while maintaining the fidelity and stability of the PDP loop.
3) **Industrial IoT under ultra-low power.** In industrial IoT systems, the PDP loop is typically energy- and loss-sensitive, as excessive computation or memory usage may lead to system degradation or data loss. To address the stringent energy and memory constraints of industrial sensors, hybrid compression pipelines are widely adopted. For instance, a recent study proposes a three-stage optimization framework integrating pruning, quantization, and kD to compress deep neural networks. This approach effectively accelerates models for deployment on low-resource industrial hardware (e.g., Raspberry Pi and microcontrollers) in real-time IoT scenarios, achieving significant compression without substantial accuracy loss [81]. From a PDP-loop viewpoint, compression directly contributes to sustaining long-term operation and reliable physical feedback.

Ultimately, these real-world cases demonstrate that there is no one-size-fits-all solution; compression strategies must be meticulously co-designed with deployment constraints. Viewed through the PDP-loop lens, application-oriented hybrid compression aligns the digital intelligence layer with physical-world constraints such as timing, reliability, and energy efficiency, ensuring that edge-deployed models can effectively support diverse cyber–physical systems. Such application-oriented hybridization ensures that deep learning

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

models remain both robust and efficient, spanning from ultra-low-power sensors to safety-critical autonomous systems.

C. Optimal Combination Strategies for Compression

Different hybrid orders result in distinct trade-offs regarding model size, latency, energy consumption, and accuracy. Hence, the choice of compression sequence must account for architectural complexity and deployment constraints. Three representative strategies are outlined below:

1) **Prune → Quantize → KD.** This pipeline represents a "compress-then-recover" paradigm, where the model is first physically compressed through pruning and quantization, and knowledge distillation is subsequently applied to restore the accuracy lost during these aggressive reductions. Pruning is chosen as the initial step because it directly reduces the model's structural complexity by removing redundant parameters or channels, which lowers FLOPs and memory requirements. Recent comprehensive studies by Chhawria et al. have investigated such compression sequences, highlighting that while this order effectively reduces model size, careful tuning is required to balance the cumulative error accumulation [82].
2) **Joint Quantization–Pruning Optimization.** Instead of sequential execution, quantization and pruning are co-optimized within one framework, typically using gradients, sensitivity metrics, or search algorithms for global balance. Motetti et al. [83] propose gradient-based joint optimization that outperforms isolated strategies in energy efficiency and accuracy for lightweight models.
3) **Prune → KD → Quantize.** Best suited for large, complex architectures. Structural pruning and distillation are integrated during training so that the student learns key features while being compressed. Here again, pruning is applied first to remove unnecessary model components, reducing the parameter space and computational load. This allows knowledge distillation to focus on essential features and ensures quantization is applied on a compact model, achieving better performance and efficiency. Quantization is applied afterward to minimize storage and computation. PDC [84] uses clustering-based reconstruction to combine pruning and KD, improving Top-1 accuracy by 0.14% on CIFAR-10 after 61.5% FLOPs reduction and even yielding "accuracy improvement after compression" on CIFAR-100.

In summary, these strategies span structural compactness, numeric efficiency, and performance recovery, providing flexible pathways to deploy LLMs effectively on edge devices.

IV. Software-Hardware Co-Design and Deployment Optimization

Deploying LLMs on resource-constrained edge devices must simultaneously address limited compute capability, tight memory budgets, energy sensitivity, and real-time inference requirements. Unlike cloud or data-center environments, edge platforms are characterized by strict power envelopes, fragmented memory hierarchies, and application-dependent latency deadlines, which fundamentally reshape system-level design objectives. As a result, relying solely on model-level compression or isolated hardware acceleration is no longer sufficient for complex edge applications. To ensure deployability and high performance, optimization is required from two complementary perspectives: (1) *software-hardware co-design*, which builds a multi-layer collaboration stack from the system level down to instruction-level support so that model algorithms, runtime frameworks, and low-level hardware capabilities are tightly integrated in a manner explicitly constrained by edge-specific resource budgets and real-time requirements, a paradigm that has been widely explored in domain-specific architectures and deep learning systems [85], [86]; and (2) *deployment pipeline optimization*, which refines model conversion [87], compilation, execution scheduling, and resource management to enable fast, stable, and efficient inference on real devices. Based on this idea, the following subsections provide an in-depth discussion of these two aspects.

A. Software-Hardware Co-Design on the Edge

With the evolution of edge computing, software–hardware co-design has become a key strategy for improving the performance of LLMs under strict resource constraints. In edge scenarios, co-design is not merely an efficiency optimization, but a necessity imposed by tight power budgets, limited on-chip memory, and stringent end-to-end latency constraints. By colocating computation, storage, and networking at edge nodes, such co-design enables near-data processing, significantly reducing latency and improving system efficiency [88]. For example, Armv9-based edge AI platforms integrate specialized NPUs (e.g., Ethos) with efficient Cortex-A CPUs to provide strong AI compute capability while maintaining low power consumption [89]. From an edge-centric perspective, software–hardware co-design can be systematically decomposed into multiple levels, each targeting a different class of resource bottlenecks, as summarized in Table VIII.

1) System-level co-design. System-level co-design focuses on global coordination between software scheduling policies and hardware resource management, especially in constrained environments. Software optimizations include resource-aware scheduling for cloud–edge collaboration, single-device inference, and lightweight runtimes for efficient memory management and tensor operations. For example, DACAPO [90] accelerates continual video analytics in self-checkout systems on edge servers by dynamically adapting training and inference strategies based on resource availability. Compared with state-of-the-art GPU-based continual learning systems Ekya [91] and EOMU [92], DACAPO improves accuracy by 6.5% and 5.5%, respectively. These results highlight that system-level co-design on the edge must explicitly account for workload dynamics, limited hardware headroom, and the need for rapid adaptation, rather than relying on static scheduling policies designed for cloud environments.

2) Model-level co-design. Model-level co-design emphasizes aligning software algorithms with specific hardware

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

TABLE VIII
SOFTWARE–HARDWARE COLLABORATIVE STRATEGIES AT DIFFERENT LEVELS

Level	Optimization Strategy	Technical Principle	Performance Gain
System-level [90]–[92]	Resource-aware scheduling	Joint optimization of software scheduling and hardware resource management	Up to 6.5% and 5.5% higher accuracy than Ekya and EOMU
Model-level [93]	Algorithm–hardware co-design	Aligning model architectures with hardware characteristics	2–3× speedup in inference
Operator-level [94]	Optimizing core computational operators	Hardware-optimized implementations of critical operators	Up to 2× improvement in model throughput
Instruction-level [95]	Instruction set extension	Designing dedicated instructions for AI workloads	3–5× energy-efficiency improvement

capabilities to enable hardware-aware inference. By carefully coupling model structures with hardware features (e.g., DVFS, mixed precision), inference efficiency can be significantly improved. EdgeBERT [93], for instance, integrates dynamic voltage/frequency scaling, pruning, and quantization to substantially enhance energy efficiency, achieving favorable accuracy–latency trade-offs on edge devices. Such model-level co-design is particularly critical on edge platforms, where dynamic power management and precision scaling are often the primary levers for meeting energy and thermal constraints under real-time inference demands.

3) Operator-level co-design. At the operator level, optimization targets core computational units (e.g., matrix multiplication, attention, convolution) and leverages specialized accelerators. The component-based roofline performance model in [94] demonstrates that, by jointly optimizing kernel fusion, memory access patterns, and operator scheduling, end-to-end speedups of 1.07× to 2.15× can be achieved, revealing substantial operator-level potential.

4) Instruction-level co-design. Instruction-level co-design further improves execution efficiency by extending the instruction set and customizing functional units. For example, [95] proposes RISC-V instruction set extensions and custom function units for sparse DNNs. Through instruction-level co-optimization for semi-structured and unstructured sparsity, the design achieves up to 3× speedup for unstructured sparsity, 4× for semi-structured sparsity, and 5× acceleration for TinyML workloads on an FPGA without significantly increasing resource usage. This form of instruction-level specialization is especially well-suited for edge devices, where modest hardware extensions can yield substantial efficiency gains without incurring the cost or complexity of full-scale accelerators.

In summary, multi-level co-design—from system scheduling to instruction customization—forms an integrated optimization stack that balances accuracy, speed, and power consumption, providing critical support for efficient deployment in resource-limited edge environments.

B. Deployment Optimization on Edge Devices

Model deployment on edge devices is a complex system engineering problem involving model compression, hardware adaptation, and application-specific requirements. The primary goal is to achieve efficient model execution under limited compute, memory, and energy resources. The deployment pipeline typically includes four stages: model conversion, compilation optimization, execution optimization, and resource management. Each stage offers targeted techniques and measurable performance gains.

1) Model conversion. This stage converts models trained in frameworks such as TensorFlow or PyTorch into lightweight representations suitable for edge inference. Core optimizations include graph simplification and format conversion [96], such as removing redundant nodes, simplifying graph topology, and fixing input shapes. Converting to a standard IR often reduces model size by 10%–20%, providing a solid basis for subsequent optimizations.

2) Compilation optimization. Compilation optimization performs deep analysis and transformation of the computation graph through operator fusion and memory optimization. Merging multiple small operators into a single kernel reduces memory accesses and kernel launch overhead. Optimizing tensor layout and memory allocation improves data locality, delivering 30%–50% speedup, particularly for sequence models and Transformers. For instance, [97] proposes fusion and scheduling rules for memory-bound operators, achieving on average 1.23× acceleration (up to 1.95×) on CPU/GPU platforms.

3) Execution optimization. Execution-level optimization leverages parallelism and dynamic scheduling to allocate resources efficiently at runtime [98], [99]. By exploiting task-level parallelism on multi-core CPUs and accelerators, and using adaptive schedulers to account for hardware load and task priority, it reduces context-switch overhead and idle time. The work in [100] introduces hybrid parallelism that combines inter-group data parallelism with intra-group pipeline parallelism, achieving 1.7–3.4× throughput improvements on heterogeneous GPU clusters without sacrificing accuracy.

4) Resource management. Resource management addresses the limited memory on edge devices via memory pooling and cache-aware strategies [101]. A memory pool pre-allocates fixed-size blocks for reuse across inference steps, avoiding frequent allocation and deallocation. The dynamic memory-swapping mechanism in [102] combines pre-allocated pools with prefetching to offload non-critical tensors to external storage when memory is insufficient, while a contiguous layout improves cache utilization. Experiments show 31%–58% memory reduction and 22%–41% latency reduction.

Overall, edge deployment optimization is a full-stack process spanning model conversion, compilation, runtime scheduling, and memory/resource management. By simplifying model structure, fusing operators, exploiting parallelism,

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

and intelligently managing memory, these stages collectively enhance inference efficiency, reduce latency, and lower resource usage, making large-scale models practical in edge scenarios.

V. Research Challenges and Future Directions

Based on the surveyed literature, the major open challenges and future research opportunities for the efficient compression and deployment of LLMs in IoT ecosystems are as follows.

A. Robust Ultra-Low-Bit Quantization

Quantization remains essential for reducing memory footprint and computational cost on resource-constrained IoT devices. However, moving toward ultra-low-bit precision (e.g., 4-bit or 1-bit) introduces substantial instability due to activation outliers, distribution shifts, and hardware-induced noise. PTQ methods frequently fail on non-i.i.d. IoT data, whereas QAT incurs expensive retraining overhead, limiting its applicability at scale [103], [104]. Furthermore, heterogeneous sensors and fluctuating environmental conditions exacerbate quantization error, resulting in inconsistent performance across devices and scenarios.

Future work may focus on *adaptive quantization mechanisms* that incorporate operator-wise mixed precision, layer-level sensitivity modeling, and learnable clipping/scale parameters. Integrating online calibration signals from runtime statistics (e.g., activation ranges, quantization noise feedback) can further enhance robustness in dynamic IoT environments. Studying hardware variation–aware quantization and noise-tolerant quantizers will also be necessary to ensure stable performance across diverse edge platforms.

B. Hardware Inefficiency of Unstructured Pruning

Although unstructured pruning achieves high sparsity ratios, its irregular sparsity patterns often lead to inefficient execution on current IoT hardware. Many embedded accelerators, DSPs, and NPUs lack fine-grained sparse kernels, causing memory access divergence and low effective speedup. This mismatch results in high theoretical compression but limited real-world gains. Moreover, existing sparse formats are not unified across toolchains and often rely on dense fallback operators, further reducing practical benefits.

To address this issue, future research may investigate pruning methods that are *co-designed with compilers and hardware backends*. This includes designing hybrid sparsity patterns that balance flexibility and hardware alignment, developing fine-grained sparse kernels optimized for IoT accelerators, and leveraging graph-level rewrites to reduce irregular memory access. Thus, compiler-assisted pruning, sparse operator fusion, and structured–unstructured mixed sparsification are promising directions for achieving both high sparsity and efficient execution.

C. Lack of Unified Framework for Hybrid Compression

Hybrid compression—combining quantization, pruning, distillation, and lightweight architectural modifications—offers superior trade-offs but currently lacks unified frameworks for comprehensive optimization. Existing toolchains apply these techniques independently, leading to inconsistent operator semantics, incompatible intermediate representations, and fragmented deployment workflows. These issues become more severe when models must be deployed across cloud–edge–device hierarchies or when multiple compressed models collaborate.

A promising research direction is to build *unified hybrid compression frameworks* that integrate design-space exploration, multi-objective optimization (accuracy, latency, energy), and hardware-aware mapping into a cohesive workflow. Such frameworks should provide standardized IR formats, unified operator abstractions, and automated deployment pipelines to ensure consistency across diverse backends. Incorporating differentiable or evolutionary search for hybrid configuration and enabling end-to-end training–inference closure will further enhance practicality.

D. Limited Cross-Architecture Generalization

IoT ecosystems encompass a wide range of heterogeneous hardware—including ARM CPUs, edge GPUs, NPUs, DSPs, MCUs, and analog or memristor-based accelerators—each with distinct operator sets, memory hierarchies, and precision characteristics. Compression strategies optimized for one device often fail to generalize across others, incurring high retuning cost or inconsistent performance [105]–[107]. This hardware diversity poses a barrier to large-scale, portable deployment of compressed models in real IoT systems.

Future research may develop *cross-architecture transferable compression strategies*, such as hardware-agnostic operator representations, meta-learned compression policies, and architecture-conditioned performance prediction models. Meanwhile, *auto-tuning runtimes* that automatically select precision configurations, sparsity patterns, and scheduling strategies for each target platform will be essential. Lightweight calibration, compilation backends with pluggable hardware profiles, and cross-device IR compatibility will further facilitate portable deployment.

E. Insufficient Robustness in Dynamic Edge Environments

IoT environments are inherently dynamic due to distribution shift, intermittent connectivity, fluctuating bandwidth, and varying compute budgets. Compressed models trained offline often fail to maintain accuracy and QoS guarantees when deployed in such time-varying conditions [108], [109]. In particular, compressed models may be vulnerable to sensor drift, incomplete inputs, environmental noise, or changing temporal patterns. Models optimized for static scenarios therefore degrade quickly in real deployments.

Addressing this challenge requires *self-adaptive dynamic inference mechanisms*, including runtime-reconfigurable subnetworks, conditional computation graphs, and dynamic sparsity/precision adjustment based on real-time resource and data conditions. Integrating environment-aware scheduling, online model selection, or dynamic cache–inference pipelines can provide stable performance under workload fluctuation. Embedding robustness objectives directly into compression, rather

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

than treating robustness as a post-hoc property, represents a promising research direction for long-term deployment stability.

VI. Conclusion

Driven by the increasing demand for edge AI, model compression has become a key enabler for deploying large AI models on resource-constrained edge devices. Although recent advances have demonstrated considerable potential, research in this field remains exploratory. Moreover, many challenges persist due to the lack of unified and comprehensive solutions. This work presents a comprehensive review of edge-oriented model compression. We first trace the evolution of edge AI and large AI models, and then comprehensively examine the application of foundational compression techniques, including quantization, pruning, and KD, in edge deployment scenarios. We also analyze the critical roles of hybrid compression strategies and software-hardware co-optimization, highlighting their importance in improving inference efficiency, reducing power consumption, and mitigating memory and computational bottlenecks on edge devices. From the perspective of real-world edge AI, we summarize the challenges that arise in edge LLM inference and training, including performance degradation under extreme compression, strict energy-latency constraints, and limited robustness in dynamic edge environments. Based on these insights, potential research directions are discussed. In summary, as the demand for on-device intelligence continues to grow, edge-oriented model compression will play an increasingly central role in enabling practical deployment of large AI models. Future research should focus on adaptive compression strategies and efficient inference/training mechanisms tailored to dynamic edge execution environments, aiming to support stable, efficient, and intelligent LLM applications in edge networks.

Acknowledgment

The above work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant U25A20436, the National Natural Science Foundation of China (NSFC) (62372047), Guangxi Key Research & Development Program (FN2504240036,2025FN96441087), Guangdong S&T Programme (No. 2025B0101120006), the Natural Science Foundation of Guangdong Province (2024A1515011323), the Supplemental Funds for Major Scientific Research Projects of Beijing Normal University, Zhuhai (ZHPT2023002), the Fundamental Research Funds for the Central Universities, Guangdong Province Educational Science Planning Research Project under 2025JKZG069, Higher Education Research Topics of Guangdong Association of Higher Education in the 14th Five-Year Plan under 24GYB207.

References

[1] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[2] Gemini Team and Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[3] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[4] Y. Wang, C. Yang, S. Lan, L. Zhu, and Y. Zhang, "End-edge-cloud collaborative computing for deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2647–2683, 2024.

[5] X. Xu, M. Li, C. Tao, T. Shen, R. Cheng, J. Li, C. Xu, D. Tao, and T. Zhou, "A survey on knowledge distillation of large language models," *arXiv preprint arXiv:2402.13116*, 2024.

[6] J. Chen, S.-h. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, and S.-H. G. Chan, "Run, don't walk: chasing higher flops for faster neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 12 021–12 031.

[7] Y. Choukroun, E. Kravchik, F. Yang, and P. Kisilev, "Low-bit quantization of neural networks for efficient inference," in *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*. IEEE, 2019, pp. 3009–3018.

[8] J. Zhou, J. Wu, Y. Gao, Y. Ding, C. Tao, B. Li, F. Tu, K.-T. Cheng, H. K.-H. So, and N. Wong, "Dybit: Dynamic bit-precision numbers for efficient quantized neural network inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1613–1617, 2023.

[9] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature medicine*, vol. 29, no. 8, pp. 1930–1940, 2023.

[10] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1556–1577, 2024.

[11] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.

[12] W. Wang, W. Chen, Y. Luo, Y. Long, Z. Lin, L. Zhang, B. Lin, D. Cai, and X. He, "Model compression and efficient inference for large language models: A survey," *arXiv preprint arXiv:2402.09748*, 2024.

[13] J. Guo, X. Chen, Y. Tang, and Y. Wang, "Slimllm: Accurate structured pruning for large language models," *arXiv preprint arXiv:2505.22689*, 2025.

[14] Z. Li, C. Huang, X. Wang, H. Hu, C. Wyeth, D. Bu, Q. Yu, W. Gao, X. Liu, and M. Li, "Lossless data compression by large models," *Nature Machine Intelligence*, pp. 1–6, 2025.

[15] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Communications Surveys & Tutorials*, 2025.

[16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[17] Y. Ma, T. Jin, X. Zheng, Y. Wang, H. Li, Y. Wu, G. Jiang, W. Zhang, and R. Ji, "Ompq: Orthogonal mixed precision quantization," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 7, 2023, pp. 9029–9037.

[18] Y. Guo, Z. Hao, J. Shao, J. Zhou, X. Liu, X. Tong, Y. Zhang, Y. Chen, W. Peng, and Z. Ma, "Pt-bitnet: Scaling up the 1-bit large language model with post-training quantization," *Neural Networks*, p. 107855, 2025.

[19] Z. Zhang, W. Shao, J. Gu, X. Wang, and P. Luo, "Differentiable dynamic quantization with mixed precision and adaptive resolution," in *International Conference on Machine Learning*. PMLR, 2021, pp. 12 546–12 556.

[20] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in neural information processing systems*, vol. 35, pp. 27 168–27 183, 2022.

[21] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, pp. 1–72, 2025.

[22] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "Llm-qat: Data-free quantization aware training for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 467–484.

[23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.

[24] G. Yang, C. He, J. Guo, J. Wu, Y. Ding, A. Liu, H. Qin, P. Ji, and X. Liu, "Llmcbench: Benchmarking large language model compression for efficient deployment," *Advances in Neural Information Processing Systems*, vol. 37, pp. 87 532–87 544, 2024.

[25] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "Edgeshard: Efficient llm inference via collaborative edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119 – 13 131, 2025.

[26] M. Lawenda, K. Khloponin, K. Samborski, and Ł. Szustak, "Profiling and optimization of multicard gpu machine learning jobs," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 18-20, p. e70196, 2025.

[27] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 8754–8782.

[28] I. Han, Z. Zhang, Z. Wang, Y. Zhu, S. Liang, J. Liu, H. Lin, M. Zhao, C. Xu, K. Wan *et al.*, "Calibquant: 1-bit kv cache quantization for multimodal llms," *arXiv preprint arXiv:2502.14882*, 2025.

[29] L. Wang, C. Zeng, M. Zhang, J. Wu, and L. Nie, "Domain aware post training quantization for vision transformers in deployment," *Pattern Recognition*, p. 112182, 2025.

[30] L. Zhu, J. Li, H. Qin, W. Li, Y. Zhang, Y. Guo, and X. Yang, "Passionsr: Post-training quantization with adaptive scale in one-step diffusion based image super-resolution," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 12 778–12 788.

[31] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, "Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization," in *European conference on computer vision*. Springer, 2022, pp. 191–207.

[32] C. Wang, Z. Wang, X. Xu, Y. Tang, J. Zhou, and J. Lu, "Q-vlm: Post-training quantization for large vision-language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 114 553–114 573, 2024.

[33] S. Li, T. Lin, L. Lin, W. Zhang, J. Liu, X. Yang, J. Li, Y. He, X. Song, J. Xiao *et al.*, "Eyecaregpt: Boosting comprehensive ophthalmology understanding with tailored dataset, benchmark and model," in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 3893–3902.

[34] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, "Efficientqat: Efficient quantization-aware training for large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 10 081–10 100.

[35] M. Navardi, R. Aalishah, Y. Fu, Y. Lin, H. Li, Y. Chen, and T. Mohsenin, "Genai at the edge: Comprehensive survey on empowering edge devices," in *Proceedings of the AAAI Symposium Series*, vol. 5, no. 1, 2025, pp. 180–187.

[36] X. Shen, P. Dong, L. Lu, and et al., "Agile-quant: Activation-guided quantization for faster inference of llms on the edge," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 18 944–18 951.

[37] S. Shen, Z. Dong, J. Ye, and et al., "Q-bert: Hessian based ultra low precision quantization of bert," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 5, 2020, pp. 8815–8821.

[38] M. W. U. Rahman, M. M. Abrar, H. G. Copening, and et al., "Quantized transformer language model implementations on edge devices," in *2023 International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2023, pp. 709–716.

[39] B. Guo, S. Tang, C. Zeng, and Z. Shen, "Mosaicdiff: Training-free structural pruning for diffusion model acceleration reflecting pretraining dynamics," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 1655–1664.

[40] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 558 – 10 578, 2024.

[41] A. Hassan, "Ai-driven urban planning: Integration paths and science communication strategies in smart city development," *Journal of Artificial Intelligence and Science Communication*, vol. 1, no. 1, pp. 41–53, 2025.

[42] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, "Coruscant: Co-designing gpu kernel and sparse tensor core to advocate unstructured sparsity in efficient llm inference," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture®*, 2025, pp. 232–245.

[43] H. Hu, P. Zhao, P. Li, and et al., "Fasp: Fast and accurate structured pruning of large language models," *arXiv preprint arXiv:2501.09412*, 2025.

[44] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang *et al.*, "A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 6, pp. 1–87, 2025.

[45] Y. Tong, W. Lu, X. Cui, Y. Mao, and Z. Zhao, "Dapt: Domain-aware prompt-tuning for multimodal fake news detection," in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 7902–7911.

[46] J. Sander, D. Moe, A. Cohen, and et al., "Constrained edge ai deployment: Fine-tuning vs distillation for llm compression," *arXiv preprint arXiv:2505.18166*, 2025.

[47] W. Huang, Y. Hu, G. Jian, and et al., "Pruning large language models with semi-structural adaptive sparse training," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 23, 2025, pp. 24 167–24 175.

[48] G. Fang, H. Yin, S. Muralidharan, and et al., "Maskllm: Learnable semi-structured sparsity for large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 7736–7758.

[49] A. Neth, "Unit pruner: Unstructured inference-time pruning for battery-free systems," *arXiv preprint arXiv:2507.07885*, 2025.

[50] I. F. Shihab, S. Akter, and A. Sharma, "Efficient unstructured pruning of mamba state-space models for resource-constrained environments," *arXiv preprint arXiv:2505.08299*, 2025.

[51] A. Belhadi, Y. Djenouri, and A. N. Belbachir, "Lightprune: Latency-aware structured pruning for efficient deep inference on embedded devices," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 1688–1697.

[52] J. C. Beard, "The sparse data reduction engine: chopping sparse data one byte at a time," in *Proceedings of the International Symposium on Memory Systems*, 2017, pp. 34–48.

[53] M. Livanos, I. Davidson, S. Wong, and et al., "Cooperative knowledge distillation: A learner agnostic approach," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 13, 2024, pp. 14 124–14 131.

[54] T. Xue, Y. Yang, and P. Zhai, "Conversion rate prediction in online advertising: modeling techniques, performance evaluation and future directions," *Information Processing & Management*, vol. 63, no. 2, p. 104450, 2026.

[55] K. Zhao, L. Li, K. Ding, N. Z. Gong, Y. Zhao, and Y. Dong, "A survey on model extraction attacks and defenses for large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 6227–6236.

[56] Z. Li, P. Xu, X. Chang, and et al., "When object detection meets knowledge distillation: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10 555–10 579, 2023.

[57] F. M. E. Alabbasy, A. S. Abohamama, and M. F. Alrahmawy, "Compressing medical deep neural network models for edge devices using knowledge distillation," *Journal of King Saud University-Computer and Information Sciences*, vol. 35, no. 7, p. 101616, 2023.

[58] E. Tanghatari, M. Kamal, A. Afzali-Kusha, and et al., "Federated learning by employing knowledge distillation on edge devices with limited hardware resources," *Neurocomputing*, vol. 531, pp. 87–99, 2023.

[59] J. Gou, B. Yu, S. J. Maybank, and et al., "Knowledge distillation: A survey," *International Journal of Computer Vision*, vol. 129, no. 6, pp. 1789–1819, 2021.

[60] J. Lee, D. Das, M. Hayat, and et al., "Customkd: Customizing large vision foundation models for edge model improvement via knowledge distillation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 25 176–25 186.

[61] J. Jang, C. Ma, and B. Lee, "VL2Lite: Task-specific knowledge distillation from large vision-language models to lightweight networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 30 073–30 083.

[62] T. Jiang, J. Hao, S. Harsha, R. D. Rachmanto, A. Setyanto, L. Ramaswamy, and I. K. Kim, "Convergo: Multi-slo-aware scheduling for heterogeneous ai accelerators on edge devices," in *2025 IEEE International Conference on Edge Computing and Communications (EDGE)*. IEEE, 2025, pp. 115–125.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

[63] J.-G. Jang, C. Quan, H. D. Lee, and U. Kang, "Falcon: lightweight and accurate convolution based on depthwise separable convolution," *Knowledge and Information Systems*, vol. 65, no. 5, pp. 2225–2249, 2023.

[64] H. Li, Z. Chen, J. Zhou, and et al., "Reducing the teacher–student gap via elastic student," in *International Conference on Knowledge Science, Engineering and Management*. Cham: Springer Nature Switzerland, 2023, pp. 442–453.

[65] Q. Xiang, M. Zhang, Y. Shang, and et al., "DKDM: Data-free knowledge distillation for diffusion models with any architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 2955–2965.

[66] G. Chen, J. Chen, F. Feng, and et al., "Unbiased knowledge distillation for recommendation," in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, 2023, pp. 976–984.

[67] S. Jain, S. Karthik, and V. Gandhi, "Simplifying knowledge transfer in pretrained models," *arXiv preprint arXiv:2510.22208*, 2025.

[68] Z. Zhang, R. Xu, C. Wang, and et al., "DFMC: Feature-driven data-free knowledge distillation," *IEEE Transactions on Circuits and Systems for Video Technology*, 2025.

[69] Y. Boo, S. Shin, and W. Sung, "Capacity of deep neural networks under parameter quantization," unpublished / publication year not found.

[70] A. Kulshrestha, X. Liu, H. Ushijima-Mwesigwa, and et al., "QadaPrune: Adaptive parameter pruning for training variational quantum circuits," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 2. IEEE, 2024, pp. 120–125.

[71] G. Habib, S. M. Kaleem, T. Rouf, and et al., "A comprehensive review of knowledge distillation in computer vision," *arXiv preprint arXiv:2404.00936*, 2024.

[72] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.

[73] M. Zhang, H. Chen, C. Shen, Z. Yang, L. Ou, X. Yu, and B. Zhuang, "Loraprune: Structured pruning meets low-rank parameter-efficient fine-tuning," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3013–3026.

[74] T. Wang, K. Liu, Y. Tang, T.-C. Hu, Z. Wang, and D. Lin, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2078–2087.

[75] J. Kim, S. Chang, and N. Kwak, "Pqk: Model compression via pruning, quantization, and knowledge distillation," in *Proc. Interspeech 2021*, 2021, pp. 4568–4572.

[76] K. Wu, H. Peng, Z. Zhou, B. Xiao, M. Liu, L. Yuan, H. Xuan, M. Valenzuela, X. S. Chen, X. Wang *et al.*, "Tinyclip: Clip distillation via affinity mimicking and weight inheritance," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 21 970–21 980.

[77] X. Chu, L. Qiao, X. Lin, S. Xu, Y. Yang, Y. Hu, F. Wei, X. Zhang, B. Zhang, X. Wei *et al.*, "Mobilevlm: A fast, strong and open vision language assistant for mobile devices," *arXiv preprint arXiv:2312.16886*, 2023.

[78] A. Wang, H. Chen, Z. Lin, J. Han, and G. Ding, "Repvit: Revisiting mobile cnn from vit perspective," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 15 909–15 920.

[79] H.-A. Rashid, A. Sarkar, A. Gangopadhyay, M. Rahnemoonfar, and T. Mohsenin, "Tinyvqa: Compact multimodal deep neural network for visual question answering on resource-constrained devices," *arXiv preprint arXiv:2404.03574*, 2024.

[80] Y. Zhang, Z. Dong, H. Yang, M. Lu, C.-C. Tseng, Y. Du, K. Keutzer, L. Du, and S. Zhang, "Qd-bev: quantization-aware view-guided distillation for multi-view 3d object detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3825–3835.

[81] S. Yarashov, O. Goziev, and S. Nurmuradova, "Efficient training of deep neural networks on resource-constrained devices for industrial applications," in *Optical and Computational Technologies for Measurements and Industrial Applications (OptiComp 2025)*, vol. 13803. SPIE, 2025, pp. 717–723.

[82] S. Chhawri, R. Mahadik, and S. Rooj, "A systematic study of compression ordering for large language models," *arXiv preprint arXiv:2511.19495*, 2025.

[83] B. A. Motetti, M. Risso, A. Burrello, and et al., "Joint pruning and channel-wise mixed-precision quantization for efficient deep neural networks," *IEEE Transactions on Computers*, 2024.

[84] T. Niu, Y. Teng, L. Jin, and et al., "Pruning-and-distillation: One-stage joint compression framework for cnns via clustering," *Image and Vision Computing*, vol. 136, p. 104743, 2023.

[85] G. De Michell and R. K. Gupta, "Hardware/software co-design," *Proceedings of the IEEE*, vol. 85, no. 3, pp. 349–365, 1997.

[86] J. Zhao, P. Zeng, G. Shen, Q. Chen, and M. Guo, "Hardware–software co-design enabling static and dynamic sparse attention mechanisms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 9, pp. 2783–2796, 2024.

[87] S. Deng, H. Zhao, W. Fang, and et al., "Edge intelligence: The confluence of edge computing and artificial intelligence," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7457–7469, 2020.

[88] Z. Liu, X. Chen, H. Wu, and et al., "Integrated sensing and edge ai: Realizing intelligent perception in 6g," *IEEE Communications Surveys & Tutorials*, 2025.

[89] Z. Ning, C. Wang, Y. Chen, and et al., "Revisiting arm debugging features: Nailgun and its defense," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 574–589, 2021.

[90] Y. Kim, C. Oh, J. Hwang, and et al., "Dacapo: Accelerating continuous learning in autonomous systems for video analytics," in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1246–1261.

[91] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, and et al., "Ekya: Continuous learning of video analytics models on edge compute servers," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 119–135.

[92] Y. Kong, P. Yang, and Y. Cheng, "Edge-assisted on-device model update for video analytics in adverse environments," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 9051–9060.

[93] T. Tambe, C. Hooper, L. Pentecost, and et al., "Edgebert: Sentence-level energy optimizations for latency-aware multi-task nlp inference," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 830–844.

[94] Y. Zhou, Z. Wang, G. Liu, and et al., "Squeezing operator performance potential for the ascend architecture," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 1156–1171.

[95] M. Sabih, A. Karim, J. Wittmann, and et al., "Hardware/software co-design of risc-v extensions for accelerating sparse dnns on fpgas," in *Proceedings of the 2024 IEEE Conference (exact venue unspecified)*, 2024, venue information incomplete.

[96] K. H. Lee, J. Y. Kwak, J. Park, and et al., "Neural network model transformation framework for on-devices," in *2023 14th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2023, pp. 1316–1318.

[97] G. Zheng, J. Li, W. Gao, and et al., "Operator fusion scheduling optimization for tvm deep learning compilers," in *2023 3rd International Symposium on Computer Technology and Information Science (ISCTIS)*. IEEE, 2023, pp. 273–277.

[98] C. Xu, J. Guo, Y. Li, H. Zou, W. Jia, and T. Wang, "Dynamic parallel multi-server selection and allocation in collaborative edge computing," *IEEE Transactions on Mobile Computing*, pp. 1–15, 2024.

[99] C. Xu, J. Guo, J. Zeng, S. Meng, X. Chu, J. Cao, and T. Wang, "Enhancing ai-generated content efficiency through adaptive multi-edge collaboration," in *The 44th IEEE International Conference on Distributed Computing Systems (ICDCS 2024)*. IEEE, 2024, pp. 1–11.

[100] C. Xu, W. Kong, M. Liu, and et al., "Hybrid parallel inference for large model on heterogeneous clusters for high throughput," in *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2023, pp. 549–554.

[101] C. Xu, J. Guo, J. Zeng, Y. Li, J. Cao, and T. Wang, "Incorporating startup delay into collaborative edge computing for superior task efficiency," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 2024, pp. 1–10.

[102] K. Wang, J. Cao, Z. Zhou, and et al., "Swapnet: Efficient swapping for dnn inference on edge ai devices beyond the memory budget," *IEEE Transactions on Mobile Computing*, vol. 23, no. 9, pp. 8935–8950, 2024.

[103] W. Huang, H. Qin, Y. Liu, and et al., "SliM-LLM: Salience-driven mixed-precision quantization for large language models," *arXiv preprint arXiv:2405.14917*, 2024.

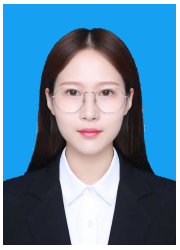
Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3675866

[104] X. Ouyang, T. Ge, T. Hartvigsen, and et al., "Low-bit quantization favors undertrained llms: Scaling laws for quantized llms with 100t training tokens," *arXiv preprint arXiv:2411.17691*, 2024.
[105] M. Wang, Y. Meng, C. Tang, and et al., "JAQ: Joint efficient architecture design and low-bit quantization with hardware-software co-exploration," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 20, 2025, pp. 21 171–21 179.
[106] K. Zhang, G. Li, N. Lu, and et al., "Hardware-aware dnn compression for homogeneous edge devices," *arXiv preprint arXiv:2501.15240*, 2025.
[107] X. Ma, G. Yuan, S. Lin, and et al., "Tiny but accurate: A pruned, quantized and optimized memristor crossbar framework for ultra efficient dnn implementation," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2020, pp. 301–306.
[108] L. Shen, A. Edalati, B. Meyer, and et al., "Robustness to distribution shifts of compressed networks for edge devices," *arXiv preprint arXiv:2401.12014*, 2024.
[109] J. Zhao, M. Zhang, C. Zeng, and et al., "LRQuant: Learnable and robust post-training quantization for large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 2240–2255.

Yuzhu Liang received his M.S. degree from National Huaqiao University of China in 2020 and his Ph.D. degree from Beijing Normal University of China in 2025. He is currently a postdoctoral researcher at Beijing Normal University. His research interests include edge computing, mobile computing, and AI.

Changfu Xu (Member, IEEE) received the B.S. degree in Communication Engineering and the M.S. degree in Software Engineering from Jiangxi University of Finance and Economics in 2015 and 2018, respectively. He received his Ph.D. degree in Computer Science and Technology from Hong Kong Baptist University, Hong Kong, China, in 2025. Currently, he is an Assistant Professor with the School of Software and IoT Engineering, Jiangxi University of Finance and Economics, Nanchang, China. His main research interests include edge computing and AIGC. He has published more than 10 papers. He won the Best Paper Runner-up Award of IEEE/ACM IWQoS 2024.



Yaxin Mei (Student Member, IEEE) received the M.S. degree from the National Huaqiao University of China, Xiamen, China, in 2021. She is currently working toward the Ph.D. degree with the Beijing Normal University of China, Beijing, China. Her research interests include mobile computing, data mining and AI.

Haodong Zou (Member, IEEE) received the B.S. degree in Software Engineering and the M.S. degree in Computer Science and Technology from Anhui University, China, in 2018 and 2021, respectively. He received his Ph.D. degree in Computer Science and Technology from Hong Kong Baptist University, Hong Kong, in 2025. Currently, he is a postdoctoral fellow at Anhui University. His main research interests include 5G mobile communication, edge AI, and multi-agent collaboration.

Jianxiong Guo (Member, IEEE) received his Ph.D. degree from the Department of Computer Science, The University of Texas at Dallas, in 2021, and his B.E. degree from the School of Chemistry and Chemical Engineering, South China University of Technology, in 2015. He is currently an associate professor at the Institute of AI and Future Networks, Beijing Normal University, China, and also at the Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, China. He is a member of IEEE/ACM/CCF. He has published more than 120 peer-reviewed papers and has served as a reviewer for numerous international journals/conferences. His research interests include edge AI, IoT, distributed machine learning, and online/combinatorial optimization.

Xinggang Fan received the Ph.D. degree from Zhejiang University, Hangzhou, China,in 2004. He is currently a Professor with the College of Zhijiang, Zhejiang University of Technology, Hangzhou, China. He has authored or coauthored more than 40 referred technical papers in proceedings and journals like TMC, TVT, INFOCOM and JCST. His main research interests include Internet of Thing, Edge Computing.

Tian Wang received his BSc and MSc degrees in Computer Science from the Central South University in 2004 and 2007, respectively. He received his PhD degree in City University of Hong Kong in in Computer Science in 2011. Currently, he is a professor in the Institute of Artificial Intelligence and Future Networks, Beijing Normal University. His research interests include internet of things, edge computing and mobile computing. He has 27 patents and has published more than 300 papers in high-level journals and conferences. He has more than 17000 citations, according to Google Scholar. His H-index is 76.

Haiyang Huang received his bachelor's degree from the School of Mathematics and Statistics, Minnan Normal University, China, in 2017 and his master's degree from the College of Computer Science and Technology, Huaqiao University, China, in 2020. He is currently a Ph.D. candidate in the School of Artificial Intelligence, Beijing Normal University, China. His research interests include mobile edge computing security, resource allocation, and edge AI.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on April 03,2026 at 05:45:36 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.